

Семинар №3

Итераторы. Продолжение

Как реализовать свой итератор?

- Перегружаем операторы `*`, `->`, `++`, `--`, ...
- Указатель не хранит указатель на сам контейнер (иначе двойной проход по указателям и проблемы с `std::swap`)
- `Const iterator` не должен быть копипастой обычного итератора. Можно использовать вспомогательную структуру `common_iterator` с шаблонным параметром `isConst`.
- Обычный итератор должен уметь каститься в константный, но не наоборот.

Reverse Iterator

- Можно получить вызовами rbegin/rend
- [std::reverse_iterator](#)
- Можно не реализовывать самому, а использовать адаптер

(back_)insert_iterator

- Для вставки элементов в контейнер используете insert_iterator/insert/insert_iterator/back_inserter/back_inserter
- В их реализации *, ++ - по op
- Не используете обычные итераторы!

Output iterators

- По этим итераторам можно записывать, но нельзя читать из них
- `(back_)insert_iterator`
- `ostream_iterator`
- Нужны для алгоритмов `copy(_if)`, `set_intersection`, ...